

# DS52

## Fondamentaux de la Science des Données

*Fiche Technique Complète — Du débutant à l'expert*

EDA · Preprocessing · Classification · Régression · Deep Learning · Big Data · XAI

**UTBM — Session 2026**

## 0. Processus Complet du Data Scientist

Le pipeline d'un projet ML suit toujours les mêmes étapes, dans cet ordre. Respecter cet ordre est crucial : sauter des étapes conduit quasi systématiquement à des erreurs coûteuses (leakage, surapprentissage, métriques trompeuses).

| Étape                   | Action                              | Point clé                                |
|-------------------------|-------------------------------------|------------------------------------------|
| 1. Compréhension métier | Définir l'objectif précis           | Prédire le churn ? Détecter une fraude ? |
| 2. Collecte des données | CSV, base de données, API, scraping | Vérifier la représentativité             |
| 3. EDA                  | Explorer, visualiser, comprendre    | Distributions, outliers, corrélations    |
| 4. Preprocessing        | Nettoyer, encoder, normaliser       | Pipeline scikit-learn                    |
| 5. Modélisation         | Choisir $\geq 3$ algorithmes        | Baseline $\rightarrow$ complexe          |
| 6. Validation           | Évaluer honnêtement                 | Cross-validation stratifiée              |
| 7. Optimisation         | Hyperparamètres                     | RandomizedSearchCV                       |
| 8. Interprétation       | Comprendre le modèle                | Feature importance, SHAP                 |
| 9. Big Data             | Adapter au volume                   | Spark, Dask, systèmes distribués         |
| 10. Déploiement         | Mettre en production                | API Flask/FastAPI, pipeline joblib       |

⚠ Règle d'or : ne jamais utiliser les données de test pour calibrer quoi que ce soit.

Toute décision (seuil, features, hyperparamètres) doit être prise sur le train set uniquement.

# 1. Types de Données & Encodage

## 1.1 Taxonomie des variables

Comprendre le type de chaque variable est la première étape obligatoire. Le type détermine l'encodage, la normalisation et les algorithmes applicables.

| Type                        | Description & exemples                                         | Traitement recommandé                                     |
|-----------------------------|----------------------------------------------------------------|-----------------------------------------------------------|
| Catégorielle nominale       | Pas d'ordre naturel. Ex : couleur, pays, job                   | One-Hot Encoding (modèles linéaires) ou Label pour arbres |
| Catégorielle ordinale       | Ordre naturel. Ex : éducation (primary < secondary < tertiary) | OrdinalEncoder avec ordre explicite                       |
| Numérique entière           | Compte discret. Ex : nombre d'appels (0, 1, 2...)              | StandardScaler ou MinMaxScaler                            |
| Numérique continue          | Valeur réelle. Ex : âge, balance, durée                        | StandardScaler, RobustScaler si outliers                  |
| Complexe — texte            | Séquences de mots. Ex : avis clients                           | TF-IDF, Word Embeddings (Word2Vec, BERT)                  |
| Complexe — image            | Matrices de pixels. Ex : radiographies                         | CNN, Transfer Learning                                    |
| Complexe — série temporelle | Courbes indexées par le temps. Ex : signal ECG                 | ARIMA, LSTM, Prophet                                      |
| Complexe — graphe           | Relations entre entités. Ex : réseau social                    | Graph Neural Networks, PageRank                           |

## 1.2 Méthodes d'encodage

### One-Hot Encoding (variables nominales)

Transforme une variable avec k catégories en k colonnes binaires. Évite d'imposer un ordre artificiel. Inconvénient : explosion dimensionnelle pour les variables à haute cardinalité (> 50 valeurs).

```
# Variable 'job' : {blue-collar, management, technician}
→ job_blue-collar  job_management  job_technician
blue-collar → 1          0          0

# sklearn
pd.get_dummies(df['job'], drop_first=True) # drop_first évite la multicollinéarité
OneHotEncoder(drop='first', sparse_output=False)
```

### Ordinal Encoding (variables ordinales)

Encode le rang naturel en entiers. Utiliser OrdinalEncoder en spécifiant l'ordre explicitement.

```
from sklearn.preprocessing import OrdinalEncoder
enc = OrdinalEncoder(categories=[['primary', 'secondary', 'tertiary']])
enc.fit_transform(df[['education']]) # → [[0], [1], [2], ...]
```

### Autres encodages utiles

| Méthode         | Usage                                                                           | Attention                                                   |
|-----------------|---------------------------------------------------------------------------------|-------------------------------------------------------------|
| Target Encoding | Remplace la catégorie par la moyenne de la cible. Bonne pour haute cardinalité. | Risque de leakage : utiliser avec cross-val ou out-of-fold. |

|                    |                                                                       |                                     |
|--------------------|-----------------------------------------------------------------------|-------------------------------------|
| Binary Encoding    | Encode en binaire. Efficace pour haute cardinalité (> 50 catégories). | Moins interprétable.                |
| Label Encoding     | Entier arbitraire. UNIQUEMENT pour les arbres.                        | Jamais pour LR ou SVM (faux ordre). |
| Frequency Encoding | Remplace par la fréquence d'apparition.                               | Peut perdre de l'information.       |
| Embedding (DL)     | Représentation dense apprise par le réseau.                           | Nécessite assez de données.         |

⚠ Ne jamais utiliser le codage label arbitraire sur une nominale (ex : rouge=1, bleu=2, vert=3) : le modèle interpréterait vert > bleu > rouge, ce qui est mathématiquement faux.

## 2. EDA — Analyse Exploratoire des Données

L'EDA est la phase de découverte : on comprend les données avant de les modéliser. Un EDA bâclé conduit presque toujours à des erreurs graves en modélisation.

### 2.1 Checklist EDA complète

| Code Python                                            | Ce qu'il révèle                                             |
|--------------------------------------------------------|-------------------------------------------------------------|
| <code>df.shape</code>                                  | Nombre de lignes et colonnes                                |
| <code>df.dtypes</code>                                 | Types de chaque colonne                                     |
| <code>df.isnull().sum()</code>                         | Valeurs manquantes par colonne                              |
| <code>df.describe()</code>                             | Stats descriptives (min, max, mean, std, quartiles)         |
| <code>df['target'].value_counts(normalize=True)</code> | Distribution de la cible — déséquilibre de classes          |
| <code>df.duplicated().sum()</code>                     | Lignes dupliquées                                           |
| <code>df.corr()</code>                                 | Matrice de corrélation numérique                            |
| <code>sns.histplot / boxplot</code>                    | Distribution + outliers par feature                         |
| <code>df.groupby('target').mean()</code>               | Valeurs moyennes par classe → signaux discriminants         |
| <code>sns.heatmap(df.corr())</code>                    | Visualisation des corrélations — détecter multicollinéarité |
| <code>df.nunique()</code>                              | Cardinalité de chaque variable catégorielle                 |

### 2.2 Identifier les outliers

#### Méthode IQR (Inter-Quartile Range)

Méthode robuste et non paramétrique. Recommandée par défaut.

```
Q1 = df['col'].quantile(0.25)
Q3 = df['col'].quantile(0.75)
IQR = Q3 - Q1
borne_inf = Q1 - 1.5 * IQR
borne_sup = Q3 + 1.5 * IQR
outliers = df[(df['col'] < borne_inf) | (df['col'] > borne_sup)]

# Exemple : balance avec Q1=72, Q3=1428 → IQR=1356 → borne_sup=3462
```

#### Méthode Z-score

Suppose une distribution normale. Sensible aux valeurs extrêmes.

```
from scipy import stats
z_scores = np.abs(stats.zscore(df[num_cols]))
df_clean = df[(z_scores < 3).all(axis=1)] # |z| > 3 = outlier
```

#### Isolation Forest (approche ML)

Méthode non paramétrique basée sur les arbres. Efficace en haute dimension.

```
from sklearn.ensemble import IsolationForest
iso = IsolationForest(contamination=0.05, random_state=42)
labels = iso.fit_predict(X) # -1 = outlier, 1 = normal
```

### 2.3 Traitement des valeurs manquantes

| Méthode                   | Quand l'utiliser                                         | Code sklearn                            |
|---------------------------|----------------------------------------------------------|-----------------------------------------|
| Suppression (dropna)      | < 5% des lignes. Risque de biais.                        | df.dropna()                             |
| Imputation par la médiane | Variables numériques asymétriques. Robuste aux outliers. | SimpleImputer(strategy='median')        |
| Imputation par la moyenne | Variables numériques symétriques.                        | SimpleImputer(strategy='mean')          |
| Imputation par le mode    | Variables catégorielles.                                 | SimpleImputer(strategy='most_frequent') |
| KNN Imputer               | Imputation basée sur les k voisins. Lent mais précis.    | KNNImputer(n_neighbors=5)               |
| MICE / IterativeImputer   | Imputation multiple par modèle itératif.                 | IterativeImputer(max_iter=10)           |
| 'unknown' comme catégorie | L'absence d'info est elle-même informative.              | df['col'].fillna('unknown')             |

⚠ TOUJOURS intégrer l'imputation dans un Pipeline scikit-learn pour éviter le data leakage !  
Calculer mean/median sur X\_train uniquement, puis appliquer sur X\_test.

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler

pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()),
    ('clf', model)
])

pipe.fit(X_train, y_train) # ✓ aucun leakage possible
```

## 2.4 Déséquilibre des classes

Problème : si 90% des données sont classe 0, un modèle prédisant toujours 0 obtient 90% d'accuracy... mais est totalement inutile. Le déséquilibre est la situation la plus fréquente en pratique (fraude, maladie rare, churn).

| Technique               | Description                                                                                                             | Code                                        |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|
| class_weight='balanced' | Poids inversement proportionnel à la fréquence. Simple et intégré à sklearn.                                            | LogisticRegression(class_weight='balanced') |
| SMOTE                   | Synthetic Minority Over-sampling. Génère des exemples synthétiques de la classe minoritaire dans l'espace des features. | from imblearn.over_sampling import SMOTE    |
| Under-sampling          | Réduire la classe majoritaire. Perd de l'information.                                                                   | RandomUnderSampler()                        |

|                  |                                                                                          |                                         |
|------------------|------------------------------------------------------------------------------------------|-----------------------------------------|
| scale_pos_weight | XGBoost uniquement :<br>= n_négatifs /<br>n_positifs.                                    | XGBClassifier(scale_pos_weight=neg/pos) |
| Threshold tuning | Ajuster le seuil de<br>décision (0.3 au lieu de<br>0.5) selon<br>precision/recall voulu. | y_pred = (y_proba > 0.3).astype(int)    |

## 3. Apprentissage Statistique — Fondements

### 3.1 Biais, Variance et compromis fondamental

Tout modèle ML commet des erreurs dues à trois sources :

```
| Erreur totale = Biais² + Variance + Bruit irréductible
```

| Source d'erreur    | Cause                                           | Symptôme                                                          | Remède                                   |
|--------------------|-------------------------------------------------|-------------------------------------------------------------------|------------------------------------------|
| Biais              | Modèle trop simple, hypothèses incorrectes      | Sous-apprentissage (underfitting) — mauvais sur train ET test     | Modèle plus complexe, plus de features   |
| Variance           | Modèle trop complexe, trop sensible aux données | Sur-apprentissage (overfitting) — bon sur train, mauvais sur test | Régularisation, plus de données, dropout |
| Bruit irréductible | Bruit inhérent aux données                      | Erreur plancher incompressible                                    | Aucun remède — accepter cette limite     |

### 3.2 Vraisemblance et Maximum de Vraisemblance (MLE)

La vraisemblance mesure à quel point un modèle (avec ses paramètres  $\theta$ ) explique les données observées  $X$ .

```
L(θ | X) = P(X | θ) ← probabilité des données sachant les paramètres

# MLE : trouver θ qui maximise L(θ|X)
# En pratique : maximiser log L(θ|X) = Σ log P(x_i|θ)
# La régression logistique utilise MLE pour estimer ses coefficients
```

### 3.3 Théorème de Bayes et apprentissage bayésien

```
P(θ | X) = P(X | θ) × P(θ) / P(X)
Posterior Vraisemblance Prior Evidence

# Prior P(θ) : nos croyances avant de voir les données
# Posterior P(θ|X) : nos croyances après avoir vu les données
# Naive Bayes : suppose l'indépendance des features
# P(y|x_1, ..., x_n) ∝ P(y) × P(x_1|y) × P(x_2|y) × ... × P(x_n|y)
```

### 3.4 Régularisation

Ajoute une pénalité pour les coefficients trop grands, réduisant le sur-apprentissage. Nécessite des données normalisées.

| Méthode        | Terme de pénalité                       | Effet                                                  | Usage                             |
|----------------|-----------------------------------------|--------------------------------------------------------|-----------------------------------|
| L1 (LASSO)     | $\lambda \sum  \beta_i $                | Pousse les coefficients vers 0 (sélection de features) | Interprétabilité, haute dimension |
| L2 (Ridge)     | $\lambda \sum \beta_i^2$                | Réduit les coefficients sans les annuler               | Multicolinéarité                  |
| Elastic Net    | $\alpha \cdot L1 + (1-\alpha) \cdot L2$ | Combine les deux                                       | Meilleur des deux mondes          |
| Dropout (DL)   | Désactive aléatoirement des neurones    | Évite la co-adaptation                                 | Réseaux de neurones               |
| Early Stopping | Arrête l'entraînement                   | Régularisation implicite                               | Gradient boosting, DL             |



quand la validation  
se dégrade

```
# Ridge (L2)
from sklearn.linear_model import Ridge
model = Ridge(alpha=1.0) # alpha =  $\lambda$ , hyperparamètre à tuner

# LASSO (L1)
from sklearn.linear_model import Lasso
model = Lasso(alpha=0.01)

# Elastic Net
from sklearn.linear_model import ElasticNet
model = ElasticNet(alpha=0.1, l1_ratio=0.5) # l1_ratio =  $\alpha$ 
```

### 3.5 Validation croisée

Évalue le modèle de manière honnête en utilisant des sous-ensembles non vus. Indispensable pour éviter l'optimisme du score sur le test set.

| Méthode             | Description                                           | Quand utiliser                      |
|---------------------|-------------------------------------------------------|-------------------------------------|
| K-Fold (k=5 ou 10)  | Divise en k groupes, k itérations train/test          | Situation standard                  |
| Stratified K-Fold   | Préserve la distribution des classes dans chaque fold | Toujours pour la classification !   |
| Leave-One-Out (LOO) | N folds pour N observations                           | Petits datasets (< 100)             |
| TimeSeriesSplit     | Folds chronologiques, ne mélange pas le futur         | Séries temporelles                  |
| Repeated K-Fold     | Répète le K-Fold plusieurs fois                       | Réduire la variance de l'estimation |

```
from sklearn.model_selection import StratifiedKFold, cross_val_score

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(pipe, X, y, cv=cv, scoring='roc_auc')
print(f'AUC = {scores.mean():.4f} ± {scores.std():.4f}')
```

## 4. Preprocessing — Préparer les données

### 4.1 Normalisation / Standardisation

La normalisation met toutes les features sur la même échelle. Indispensable pour les modèles sensibles aux distances (LR, SVM, KNN, réseaux de neurones). Inutile pour les arbres de décision.

| Méthode              | Formule                             | Résultat                | Usage                                                                     |
|----------------------|-------------------------------------|-------------------------|---------------------------------------------------------------------------|
| StandardScaler       | $(x - \mu) / \sigma$                | Moyenne 0, écart-type 1 | LR, SVM, PCA, KNN, réseaux de neurones                                    |
| MinMaxScaler         | $(x - \min) / (\max - \min)$        | Range [0, 1]            | Réseaux de neurones (ReLU), distribution non gaussienne                   |
| RobustScaler         | $(x - \text{médiane}) / \text{IQR}$ | Robuste aux outliers    | Quand les outliers sont présents                                          |
| MaxAbsScaler         | $x / \max( x )$                     | Range [-1, 1]           | Données creuses (sparse)                                                  |
| Aucune normalisation | —                                   | —                       | Random Forest, XGBoost, arbres — invariants aux transformations monotones |

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)    # fit sur train uniquement !
X_test_scaled = scaler.transform(X_test)         # transform seulement sur test
```

### 4.2 Feature Engineering

Créer de nouvelles variables à partir des variables existantes. Souvent plus impactant que le choix du modèle.

- **Interactions** : age × campaign = interaction entre âge et nombre d'appels
- **Ratios** : balance / age = solde par année de vie
- **Transformations log** :  $\log(\text{balance} + 1)$  pour réduire l'asymétrie
- **Binarisation** : was\_contacted\_before = (pdays != -1).astype(int)
- **Agrégations** : moyenne du groupe, médiane par catégorie
- **Décalages temporels** : lag features pour les séries temporelles

```
# Exemples de feature engineering
df['was_contacted_before'] = (df['pdays'] != -1).astype(int)
df['log_balance'] = np.log1p(df['balance'].clip(lower=0))
df['age_campaign_ratio'] = df['age'] / (df['campaign'] + 1)
df['negative_balance'] = (df['balance'] < 0).astype(int)
```

### 4.3 Réduction de Dimensionnalité

#### PCA — Principal Component Analysis

Projette les données sur des axes orthogonaux qui maximisent la variance. Non supervisé. Permet de réduire la dimensionnalité tout en conservant le maximum d'information.

```
from sklearn.decomposition import PCA

# Garder 95% de la variance expliquée
pca = PCA(n_components=0.95)
X_pca = pca.fit_transform(X_scaled)

# Voir la variance expliquée
print(pca.explained_variance_ratio_)    # ex : [0.52, 0.21, 0.14, ...]
print(pca.n_components_)               # nombre de composantes retenues
```

## LDA — Linear Discriminant Analysis

Méthode supervisée qui maximise la séparation entre classes. Suppose des distributions gaussiennes avec variances égales.

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
lda = LinearDiscriminantAnalysis(n_components=2) # max = n_classes - 1
X_lda = lda.fit_transform(X_scaled, y)
```

## t-SNE et UMAP (visualisation)

Méthodes non linéaires de réduction à 2D/3D pour la visualisation. Ne pas utiliser pour l'entraînement (non déterministe et lent).

```
from sklearn.manifold import TSNE
tsne = TSNE(n_components=2, perplexity=30, random_state=42)
X_2d = tsne.fit_transform(X_scaled)

# UMAP (plus rapide, meilleure préservation de structure globale)
# pip install umap-learn
import umap
reducer = umap.UMAP(n_components=2, random_state=42)
X_2d = reducer.fit_transform(X_scaled)
```

## 5. Modèles de Classification

### 5.1 Régression Logistique

Modèle linéaire pour la classification binaire. Estime la probabilité  $P(y=1|x)$  via une fonction sigmoïde. Baseline incontournable.

```
# Fonction de décision
P(Y=1) = 1 / (1 + exp(-(β₀ + β₁x₁ + ... + βₙxₙ)))

# Les coefficients s'interprètent comme des log-odds :
# e^{βᵢ} = multiplicateur de l'odds ratio pour une augmentation de 1 unité de xᵢ

from sklearn.linear_model import LogisticRegression
lr = LogisticRegression(C=1.0, class_weight='balanced', max_iter=1000)
# C = 1/λ (inverse de la régularisation L2)

# Interpréter les coefficients
odds_ratios = np.exp(lr.coef_[0])
pd.Series(odds_ratios, index=feature_names)
```

✓ Hypothèses LR : relation linéaire entre X et log-odds de Y, observations indépendantes, faible multicollinéarité, données normalisées.

### 5.2 Support Vector Machine (SVM)

Cherche l'hyperplan qui maximise la marge entre les classes. Efficace en haute dimension (texte, génomique).

| Kernel         | Formule                      | Usage                                                              |
|----------------|------------------------------|--------------------------------------------------------------------|
| Linéaire       | $x \cdot x'$                 | Données linéairement séparables, haute dimension                   |
| RBF (Gaussien) | $\exp(-\gamma \ x - x'\ ^2)$ | Données non linéaires. Paramètre $\gamma$ contrôle la flexibilité. |
| Polynomial     | $(x \cdot x' + r)^d$         | Relations polynomiales                                             |

```
from sklearn.svm import SVC
svm = SVC(kernel='rbf', C=1.0, gamma='scale', probability=True)
# C : pénalité pour les erreurs. Grand C = peu de marge, moins de régularisation
# gamma : 'scale' = 1/(n_features * X.var())

# TOUJOURS normaliser avant SVM
from sklearn.pipeline import Pipeline
pipe = Pipeline([('scaler', StandardScaler()), ('svm', svm)])
```

### 5.3 K-Nearest Neighbors (KNN)

Méthode non paramétrique : prédit en se basant sur les k voisins les plus proches. Simple mais lent pour les grands datasets.

```
from sklearn.neighbors import KNeighborsClassifier

# Distance de Minkowski : d(a,b) = (Σ|aᵢ - bᵢ|^p)^(1/p)
# p=1 → Manhattan, p=2 → Euclidienne

knn = KNeighborsClassifier(n_neighbors=5, metric='euclidean')
# Choisir k par cross-validation
# Toujours normaliser avant KNN (distances sensibles aux échelles)
```

### 5.4 Arbres de Décision (CART)

Partitionne l'espace des features récursivement en minimisant l'impureté. Très interprétable, mais tend au sur-apprentissage.

| Mesure d'impureté        | Formule                             | Usage                                            |
|--------------------------|-------------------------------------|--------------------------------------------------|
| Gini                     | $1 - \sum \hat{p}_i^2$              | Par défaut dans sklearn. Légèrement plus rapide. |
| Entropie (Cross-Entropy) | $-\sum \hat{p}_i \log_2(\hat{p}_i)$ | Similaire à Gini, parfois légèrement meilleure.  |
| MSE (régression)         | $\sum (y_i - \bar{y})^2$            | Pour les arbres de régression.                   |

```
from sklearn.tree import DecisionTreeClassifier
tree = DecisionTreeClassifier(
    max_depth=5,          # Limiter la profondeur pour éviter l'overfitting
    min_samples_leaf=20,  # Feuilles avec au moins 20 exemples
    criterion='gini',
    random_state=42
)

# Visualiser
from sklearn.tree import export_text
print(export_text(tree, feature_names=list(X.columns)))
```

## 5.5 Random Forest

Ensemble de N arbres entraînés sur des sous-ensembles aléatoires des données (bagging) ET des features. Réduit la variance sans augmenter le biais.

```
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(
    n_estimators=200,      # 100-500 arbres selon dataset
    max_depth=None,        # Arbres profonds - variance réduite par agrégation
    max_features='sqrt',   # Sous-ensemble de features par split
    class_weight='balanced',
    n_jobs=-1,             # Paralléliser sur tous les CPU
    random_state=42
)

# Feature importance (moyenne de la réduction d'impureté)
fi = pd.Series(rf.feature_importances_, index=X.columns).sort_values(ascending=False)
```

## 5.6 Gradient Boosting (XGBoost, LightGBM, CatBoost)

Construit des arbres séquentiellement, chaque arbre corrigeant les erreurs du précédent. Meilleure performance sur données tabulaires.

```
from xgboost import XGBClassifier
xgb = XGBClassifier(
    n_estimators=500,
    learning_rate=0.05,    # Plus petit = plus robuste, plus lent
    max_depth=6,
    subsample=0.8,         # Bagging par iteration
    colsample_bytree=0.8,  # Bagging des features
    scale_pos_weight=neg/pos, # Déséquilibre
    early_stopping_rounds=50, # Stopper si pas d'amélioration
    eval_metric='auc',
    random_state=42
)

# LightGBM (plus rapide, meilleur sur grands datasets)
import lightgbm as lgb
lgbm = lgb.LGBMClassifier(n_estimators=500, learning_rate=0.05, num_leaves=31)
```

```
# CatBoost (gère nativement les catégorielles)
from catboost import CatBoostClassifier
cat = CatBoostClassifier(iterations=500, cat_features=['job', 'education'])
```

## 5.7 Naive Bayes

---

Classifie en utilisant le théorème de Bayes avec l'hypothèse (naïve) d'indépendance des features. Très rapide, bon en NLP.

```
from sklearn.naive_bayes import GaussianNB, MultinomialNB, BernoulliNB

# GaussianNB : features continues (suppose distribution normale)
# MultinomialNB : fréquences (comptage de mots), NLP
# BernoulliNB : features binaires

gnb = GaussianNB()
gnb.fit(X_train, y_train)
```

## 6. Métriques d'Évaluation

### 6.1 Matrice de confusion

|              | Prédit Positif                                    | Prédit Négatif                                    |
|--------------|---------------------------------------------------|---------------------------------------------------|
| Réel Positif | TP — Vrai Positif ( $1-\beta$ )                   | FN — Faux Négatif ( $\beta$ ) = erreur de Type II |
| Réel Négatif | FP — Faux Positif ( $\alpha$ ) = erreur de Type I | TN — Vrai Négatif ( $1-\alpha$ )                  |

### 6.2 Métriques de classification

| Métrique             | Formule                                                                               | Quand utiliser                                           |
|----------------------|---------------------------------------------------------------------------------------|----------------------------------------------------------|
| Accuracy             | $TP+TN / (TP+TN+FP+FN)$                                                               | Classes équilibrées uniquement                           |
| Precision            | $TP / (TP+FP)$                                                                        | Minimiser les faux positifs (spam, pub, recommandations) |
| Recall (Sensitivité) | $TP / (TP+FN)$                                                                        | Minimiser les faux négatifs (cancer, fraude, anomalie)   |
| F1-Score             | $2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$ | Compromis precision/recall                               |
| AUC-ROC              | Aire sous la courbe TPR vs FPR                                                        | Ranking des probabilités, déséquilibre de classes        |
| Log-Loss             | $-1/n \sum [y_i \log(\hat{p}_i) + (1-y_i) \log(1-\hat{p}_i)]$                         | Calibration des probabilités                             |
| MCC                  | $(TP \times TN - FP \times FN) / \sqrt{((TP+FP)(TP+FN)(TN+FP)(TN+FN))}$               | Meilleure métrique binaire générale                      |

```
from sklearn.metrics import (classification_report, roc_auc_score,
                             confusion_matrix, f1_score, log_loss)

print(classification_report(y_test, y_pred))
auc = roc_auc_score(y_test, y_proba[:, 1])

# Courbe ROC
from sklearn.metrics import RocCurveDisplay
RocCurveDisplay.from_estimator(model, X_test, y_test)

# Courbe Precision-Recall (meilleure pour les données très déséquilibrées)
from sklearn.metrics import PrecisionRecallDisplay
PrecisionRecallDisplay.from_estimator(model, X_test, y_test)
```

### 6.3 Métriques de régression

| Métrique       | Formule                        | Interprétation                                                |
|----------------|--------------------------------|---------------------------------------------------------------|
| MSE            | $1/n \sum (y_i - \hat{y}_i)^2$ | Pénalise fortement les grandes erreurs                        |
| RMSE           | $\sqrt{\text{MSE}}$            | Même unité que la cible                                       |
| MAE            | $1/n \sum  y_i - \hat{y}_i $   | Plus robuste aux outliers                                     |
| R <sup>2</sup> | $1 - \text{SSE}/\text{SST}$    | Proportion de variance expliquée. 1 = parfait, 0 = modèle nul |

|                       |                                        |                                       |
|-----------------------|----------------------------------------|---------------------------------------|
| R <sup>2</sup> ajusté | $1 - (1 - R^2)(N - 1) / (N - p - 1)$   | Pénalise l'ajout de features inutiles |
| MAPE                  | $100 / n \sum  y_i - \hat{y}_i  / y_i$ | Erreur relative en pourcentage        |



## 7. Modèles de Régression

### 7.1 Régression Linéaire (OLS)

Modèle linéaire qui minimise la somme des carrés des résidus (SSE) pour trouver les coefficients  $\beta$ .

```
# Équation :  $\hat{y} = \beta_0 + \beta_1 x_1 + \dots + \beta_n x_n + \varepsilon$ 
# Solution analytique :  $\hat{\beta} = (X^T X)^{-1} X^T y$ 

from sklearn.linear_model import LinearRegression
lr = LinearRegression()
lr.fit(X_train, y_train)
print(f'R² = {lr.score(X_test, y_test):.4f}')
print(f'Coefficients : {dict(zip(X.columns, lr.coef_))}')
```

**Hypothèses (à vérifier) :**

- Relation linéaire entre X et y
- Résidus normalement distribués
- Homoscédasticité : variance des résidus constante
- Observations indépendantes
- Faible multicollinéarité ( $VIF < 10$ )

### 7.2 Régression Ridge, LASSO, Elastic Net

```
# LASSO : sélection automatique de features (coefficients  $\rightarrow 0$ )
from sklearn.linear_model import LassoCV
lasso = LassoCV(cv=5) # Cherche automatiquement le meilleur alpha

# Ridge : meilleur quand toutes les features contribuent
from sklearn.linear_model import RidgeCV
ridge = RidgeCV(alphas=[0.01, 0.1, 1, 10, 100])

# Elastic Net : combine les deux
from sklearn.linear_model import ElasticNetCV
en = ElasticNetCV(l1_ratio=[0.1, 0.5, 0.9], cv=5)
```

### 7.3 Régression Polynomiale

```
# Ajoute des termes polynomiaux :  $x \rightarrow x, x^2, x^3$ 
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly.fit_transform(X)

# Pipeline complet
pipe = Pipeline([
    ('poly', PolynomialFeatures(degree=2)),
    ('scaler', StandardScaler()),
    ('lr', LinearRegression())
])
```

### 7.4 Gaussian Process Regression (GPR)

Modèle bayésien non-paramétrique qui retourne une distribution de fonctions. Quantifie l'incertitude de prédiction. Idéal quand les données sont rares et l'incertitude importante.

```
# Prior :  $f(x) \sim GP(m(x), k(x, x'))$ 
# m(x) : fonction de moyenne (souvent 0)
# k(x, x') : fonction de covariance (kernel)

# Kernel RBF :  $k(x, x') = \sigma^2 \exp(-||x-x'||^2 / (2\ell^2))$ 
#  $\sigma^2$  : variance de signal,  $\ell$  : longueur de corrélation
```

```
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import RBF, WhiteKernel

kernel = RBF(length_scale=1.0) + WhiteKernel(noise_level=0.1)
gpr = GaussianProcessRegressor(kernel=kernel, n_restarts_optimizer=10)
gpr.fit(X_train, y_train)
y_mean, y_std = gpr.predict(X_test, return_std=True)
# y_std = incertitude : grande si peu de données dans cette région
```

⚠ GPR : complexité  $O(n^3)$  — ne passe pas à l'échelle pour  $n > 10\,000$ . Utiliser approximations (GPpyTorch, Sparse GP) pour les grands datasets.

## 8. Deep Learning — Réseaux de Neurones

### 8.1 Fondements

Un réseau de neurones est composé de couches de perceptrons. Chaque neurone calcule une combinaison linéaire de ses entrées, puis applique une fonction d'activation non linéaire.

```
# Perceptron :  $z = w_1x_1 + w_2x_2 + \dots + b$ , sortie = activation(z)

# Fonctions d'activation
Sigmoid :  $\sigma(z) = 1/(1+e^{-z}) \rightarrow [0,1]$  - sortie binaire
Tanh :  $\tanh(z) = (e^z - e^{-z}) / (e^z + e^{-z}) \rightarrow [-1,1]$ 
ReLU :  $\max(0, z)$   $\rightarrow [0, \infty)$  - couches cachées (défaut)
Leaky ReLU:  $\max(0.01z, z)$   $\rightarrow$  évite le problème des neurones morts
Softmax :  $e^{z_i} / \sum e^{z_j}$   $\rightarrow [0,1]$  avec somme=1 - classification multi-classe
GELU :  $z \cdot \Phi(z)$   $\rightarrow$  utilisé dans BERT, GPT
```

### 8.2 Entraînement : Gradient Descent & Backpropagation

L'entraînement consiste à minimiser la fonction de perte par descente de gradient. La rétropropagation calcule les gradients de manière efficace via la règle des dérivées en chaîne.

```
# Algorithme
1. Initialiser les poids W aléatoirement (near-zero)
2. Répéter jusqu'à convergence :
  a. Forward pass : calculer  $\hat{y} = f(X; W)$ 
  b. Calculer la perte  $L(y, \hat{y})$ 
  c. Backward pass : calculer  $\partial L / \partial W$  par rétropropagation
  d. Mettre à jour :  $W \leftarrow W - \gamma \times \partial L / \partial W$ 

# Variantes de Gradient Descent
Batch GD : utilise tout le dataset  $\rightarrow$  stable mais lent
Mini-batch : sous-ensembles de 32-512  $\rightarrow$  meilleur compromis (default)
SGD : 1 exemple par pas  $\rightarrow$  bruyant mais peut sortir des minima locaux

# Optimiseurs adaptatifs (recommandés)
Adam : adapte lr par paramètre. Défaut pour la plupart des cas.
AdaGrad : adapte lr aux gradients passés. Bon pour données creuses.
RMSprop : adapte lr via moyenne exponentielle. Bon pour RNN.
```

### 8.3 Fonctions de perte

| Tâche                       | Fonction de perte         | Formule                                           |
|-----------------------------|---------------------------|---------------------------------------------------|
| Classification binaire      | Binary Cross-Entropy      | $-[y \cdot \log(p) + (1-y) \cdot \log(1-p)]$      |
| Classification multi-classe | Categorical Cross-Entropy | $-\sum y_i \cdot \log(p_i)$                       |
| Régression                  | MSE                       | $\sum (y_i - \hat{y}_i)^2 / n$                    |
| Régression robuste          | MAE / Huber Loss          | $ y_i - \hat{y}_i $ — moins sensible aux outliers |
| Multi-label                 | Sigmoid + BCE par label   | Chaque label indépendant                          |

### 8.4 Réseaux Denses (MLP) avec PyTorch

```
import torch
import torch.nn as nn
import torch.optim as optim

# Définir l'architecture
```

```

class MLP(nn.Module):
    def __init__(self, input_dim, hidden_dims, output_dim):
        super().__init__()
        layers = []
        prev_dim = input_dim
        for h in hidden_dims:
            layers += [nn.Linear(prev_dim, h), nn.ReLU(), nn.Dropout(0.3)]
            prev_dim = h
        layers.append(nn.Linear(prev_dim, output_dim))
        self.net = nn.Sequential(*layers)

    def forward(self, x):
        return self.net(x)

model = MLP(input_dim=20, hidden_dims=[128, 64, 32], output_dim=1)
optimizer = optim.Adam(model.parameters(), lr=1e-3)
criterion = nn.BCEWithLogitsLoss()

# Boucle d'entraînement
for epoch in range(100):
    model.train()
    optimizer.zero_grad()
    output = model(X_train_tensor).squeeze()
    loss = criterion(output, y_train_tensor)
    loss.backward()
    optimizer.step()

```

## 8.5 Réseaux de Neurones Convolutifs (CNN)

Les CNN extraient des features hiérarchiques à partir de données structurées (images, texte, séries temporelles). La convolution partage les poids sur toute l'entrée (translation invariance).

| Couche     | Rôle                                                          | Paramètre clé           |
|------------|---------------------------------------------------------------|-------------------------|
| Conv2D     | Applique des filtres pour extraire des features locales       | kernel_size, n_filters  |
| ReLU       | Non-linéarité après convolution                               | —                       |
| BatchNorm  | Normalise les activations — stabilise l'entraînement          | momentum                |
| MaxPooling | Réduit la dimension spatiale, invariance aux petits décalages | pool_size               |
| Dropout    | Régularisation : désactive aléatoirement des neurones         | p = probabilité de drop |
| Flatten    | Transforme la feature map en vecteur 1D                       | —                       |
| Dense (FC) | Classification ou régression finale                           | n_units                 |

```

# Architecture CNN classique (PyTorch)
class SimpleCNN(nn.Module):
    def __init__(self, n_classes=10):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1), # 3 canaux RGB → 32 filtres
            nn.BatchNorm2d(32), nn.ReLU(),
            nn.MaxPool2d(2), # divise par 2
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64), nn.ReLU(),
            nn.MaxPool2d(2),
        )
        self.classifier = nn.Sequential(

```

```

        nn.Flatten(),
        nn.Linear(64*8*8, 256), nn.ReLU(), nn.Dropout(0.5),
        nn.Linear(256, n_classes)
    )
    def forward(self, x):
        return self.classifier(self.features(x))

```

## 8.6 Transfer Learning

Réutiliser un modèle pré-entraîné sur une grande base (ImageNet, Wikipedia) et l'adapter à sa propre tâche. C'est l'approche de référence quand les données sont limitées.

```

# Exemple avec torchvision (classification d'images)
from torchvision import models
import torch.nn as nn

# Charger ResNet50 pré-entraîné sur ImageNet
model = models.resnet50(pretrained=True)

# Option 1 : Fine-tuning complet - décommenter pour débloquer toutes les couches
# for param in model.parameters(): param.requires_grad = True

# Option 2 : Geler toutes les couches sauf la dernière (feature extraction)
for param in model.parameters():
    param.requires_grad = False

# Remplacer la tête de classification
model.fc = nn.Sequential(
    nn.Linear(2048, 256), nn.ReLU(), nn.Dropout(0.3),
    nn.Linear(256, n_classes)
)

# En NLP (Hugging Face Transformers)
from transformers import AutoModelForSequenceClassification
model = AutoModelForSequenceClassification.from_pretrained(
    'camembert-base', # Modèle BERT français
    num_labels=2
)

```

## 8.7 Réseaux Récurrents (RNN, LSTM, GRU)

Conçus pour les données séquentielles (texte, séries temporelles, audio). Maintiennent un état caché qui capture l'information des étapes précédentes.

```

# Problème des gradients qui disparaissent/explosent dans les RNN simples
# Solution : LSTM (Long Short-Term Memory) avec cellules de mémoire

# LSTM PyTorch
class LSTMClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, n_layers, n_classes):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.lstm = nn.LSTM(embed_dim, hidden_dim, n_layers,
                             batch_first=True, dropout=0.3, bidirectional=True)
        self.fc = nn.Linear(hidden_dim*2, n_classes) # *2 car bidirectionnel

    def forward(self, x):
        embedded = self.embedding(x)
        out, (h_n, c_n) = self.lstm(embedded)
        return self.fc(out[:, -1, :]) # Prendre le dernier état

# GRU : plus simple que LSTM, souvent performances similaires
self.gru = nn.GRU(embed_dim, hidden_dim, n_layers, batch_first=True)

```



## 9. Traitement du Langage Naturel (NLP)

### 9.1 Pipeline de preprocessing NLP

1. Tokenisation : découper le texte en tokens (mots, sous-mots)
2. Normalisation : minuscules, suppression de la ponctuation
3. Suppression des stopwords : enlever les mots sans valeur sémantique (le, la, de, ...)
4. Lemmatisation : réduire au lemme (mangent → manger)
5. Stemming : réduire à la racine sans contexte (ended → end) — plus bruyant que la lemmatisation

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

lemmatizer = WordNetLemmatizer()
stop_words = set(stopwords.words('french'))

def preprocess(text):
    tokens = word_tokenize(text.lower())
    tokens = [lemmatizer.lemmatize(t) for t in tokens
               if t.isalpha() and t not in stop_words]
    return tokens
```

### 9.2 Représentations vectorielles

| Méthode                   | Description                                   | Avantages / Limites                                              |
|---------------------------|-----------------------------------------------|------------------------------------------------------------------|
| Bag of Words (BoW)        | Vecteur de fréquences de mots                 | Simple. Ignore l'ordre et le sens.                               |
| TF-IDF                    | $TF \times \log(N/df)$ — poids les mots rares | Bon pour la recherche d'information.                             |
| Word2Vec (CBOW/Skip-gram) | Embedding dense appris sur contexte local     | Capture les relations sémantiques (roi - homme + femme ≈ reine). |
| GloVe                     | Embedding basé sur co-occurrence globale      | Combine local et global.                                         |
| FastText                  | Word2Vec + sous-mots                          | Gère les mots hors vocabulaire.                                  |
| BERT / CamemBERT          | Transformer bidirectionnel pré-entraîné       | État de l'art. Contextuel. Lourd.                                |

```
# TF-IDF
from sklearn.feature_extraction.text import TfidfVectorizer
tfidf = TfidfVectorizer(max_features=10000, ngram_range=(1,2))
X_tfidf = tfidf.fit_transform(corpus)

# Word2Vec (gensim)
from gensim.models import Word2Vec
model = Word2Vec(sentences, vector_size=100, window=5, min_count=2)
vector = model.wv['chat'] # vecteur du mot 'chat'

# BERT avec Hugging Face
from transformers import AutoTokenizer, AutoModel
tokenizer = AutoTokenizer.from_pretrained('camembert-base')
tokens = tokenizer(texts, padding=True, truncation=True, return_tensors='pt')
```

## 10. Big Data — Traitement à Grande Échelle

### 10.1 Quand parle-t-on de Big Data ?

Les 5V du Big Data définissent les défis posés par les grandes masses de données :

| Dimension | Description                                                      | Exemple                                                   |
|-----------|------------------------------------------------------------------|-----------------------------------------------------------|
| Volume    | Quantité de données (TB, PB)                                     | Logs de serveur, transactions bancaires                   |
| Vélocité  | Vitesse de génération et de traitement                           | Flux Twitter, capteurs IoT temps réel                     |
| Variété   | Diversité des formats (structuré, semi-structuré, non-structuré) | CSV, JSON, images, vidéos                                 |
| Véracité  | Qualité et fiabilité des données                                 | Données manquantes, erreurs de capteurs                   |
| Valeur    | Pertinence pour l'objectif métier                                | Toutes les données ne valent pas la peine d'être traitées |

### 10.2 Apache Spark

Framework de traitement distribué en mémoire. 100x plus rapide que MapReduce pour les workloads itératifs (ML). PySpark est l'API Python officielle.

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName('DS52').getOrCreate()

# Lire des données
df = spark.read.csv('s3://bucket/data.csv', header=True, inferSchema=True)

# Opérations SQL-like (lazy evaluation : rien n'est calculé jusqu'à une action)
df_filtered = df.filter(df['age'] > 30)          # Transformation (lazy)
df_grouped = df.groupBy('job').mean('balance')  # Transformation (lazy)
df_grouped.show()                             # Action (déclenche le calcul)

# SQL direct
df.createOrReplaceTempView('bank')
result = spark.sql('SELECT job, AVG(balance) FROM bank GROUP BY job')

# Machine Learning avec MLlib
from pyspark.ml import Pipeline
from pyspark.ml.classification import RandomForestClassifier
from pyspark.ml.feature import VectorAssembler, StandardScaler

assembler = VectorAssembler(inputCols=['age', 'balance', 'duration'], outputCol='features')
rf = RandomForestClassifier(numTrees=100, labelCol='label')
pipe = Pipeline(stages=[assembler, rf])
model = pipe.fit(train_df)
```

### 10.3 Dask — Pandas à grande échelle

Dask parallélise automatiquement les opérations Pandas. Idéal pour les datasets qui ne tiennent pas en mémoire (10 GB - 1 TB). API quasi-identique à Pandas.

```
import dask.dataframe as dd

# Lire un dataset volumineux (parallélisé sur plusieurs fichiers)
ddf = dd.read_csv('data_*.csv')  # lit tous les fichiers correspondants

# Mêmes opérations que Pandas
```



```

result = ddf.groupby('job')['balance'].mean().compute() # compute() déclenche

# Dask Array (NumPy distribué)
import dask.array as da
x = da.from_zarr('large_array.zarr') # Array de 500 GB
mean = x.mean().compute()

# ML avec Dask-ML
from dask_ml.model_selection import train_test_split
from dask_ml.preprocessing import StandardScaler

```

## 10.4 Architectures Big Data

| Couche              | Outils                                | Rôle                                                |
|---------------------|---------------------------------------|-----------------------------------------------------|
| Ingestion           | Kafka, Flume, Kinesis                 | Collecte et streaming de données en temps réel      |
| Stockage            | HDFS, S3, Azure Blob, Delta Lake      | Stockage distribué et résilient                     |
| Processing batch    | Spark, Hive, Flink                    | Traitement de grands volumes de données historiques |
| Processing stream   | Spark Streaming, Kafka Streams, Flink | Traitement temps réel (faible latence)              |
| Data Warehouse      | BigQuery, Redshift, Snowflake         | Stockage analytique optimisé pour les requêtes SQL  |
| ML à grande échelle | MLflow, Ray, Horovod                  | Entraînement distribué, tracking des expériences    |
| Orchestration       | Airflow, Prefect, Luigi               | Planification et monitoring des pipelines           |

## 10.5 Optimisation pour grands datasets avec scikit-learn

```

# Partial fit : entraînement par mini-batches (out-of-core learning)
from sklearn.linear_model import SGDClassifier
sgd = SGDClassifier(loss='log_loss') # équivalent LR mais en ligne

for chunk in pd.read_csv('huge_file.csv', chunksize=10000):
    X_chunk = chunk.drop('target', axis=1)
    y_chunk = chunk['target']
    sgd.partial_fit(X_chunk, y_chunk, classes=[0,1])

# Réduire la mémoire avec les types
df['age'] = df['age'].astype('int8') # au lieu de int64 (8x moins de mémoire)
df['balance'] = df['balance'].astype('float32')

# Parquet : format colonnaire compressé (10-100x moins de mémoire que CSV)
df.to_parquet('data.parquet') # écriture
df = pd.read_parquet('data.parquet') # lecture très rapide

```

## 11. XAI — Explicabilité des Modèles (Interprétable AI)

### 11.1 Pourquoi l'explicabilité ?

Les modèles complexes (Random Forest, XGBoost, réseaux de neurones) sont des boîtes noires. L'XAI permet de comprendre et d'auditer leurs décisions — obligation légale (RGPD) pour les décisions automatisées impactant des personnes.

### 11.2 Feature Importance (globale)

```
# 1. Importance par impureté (Random Forest, XGBoost) – rapide mais biaisé
fi = pd.Series(rf.feature_importances_, index=X.columns).sort_values(ascending=False)
fi.plot(kind='barh')

# 2. Permutation Importance – moins biaisée
from sklearn.inspection import permutation_importance
r = permutation_importance(model, X_test, y_test, n_repeats=10, random_state=42)
pi = pd.Series(r.importances_mean, index=X.columns).sort_values(ascending=False)
```

### 11.3 SHAP — SHapley Additive exPlanations

SHAP est la méthode d'interprétabilité la plus complète. Elle calcule la contribution marginale de chaque feature à chaque prédiction, en se basant sur la théorie des jeux coopératifs (valeurs de Shapley).

```
# Propriété clé : sum(SHAP values) = prédiction - valeur_moyenne

import shap

# TreeExplainer (optimisé pour les arbres : RF, XGBoost, LightGBM)
explainer = shap.TreeExplainer(xgb_model)
shap_values = explainer.shap_values(X_test)

# Importance globale (summary plot)
shap.summary_plot(shap_values, X_test, feature_names=X.columns)

# Explication locale (une prédiction)
shap.waterfall_plot(shap.Explanation(
    values=shap_values[0], base_values=explainer.expected_value,
    data=X_test.iloc[0], feature_names=X.columns
))

# Dependence plot : effet d'une feature sur les SHAP values
shap.dependence_plot('balance', shap_values, X_test)

# Pour les modèles non-arbres (KernelExplainer – plus lent)
explainer = shap.KernelExplainer(model.predict_proba, X_train[:100])
shap_values = explainer.shap_values(X_test[:50])
```

### 11.4 LIME — Local Interpretable Model-agnostic Explanations

LIME explique une prédiction individuelle en entraînant un modèle linéaire simple localement autour de cette instance. Agnostique au modèle (fonctionne avec n'importe quel modèle).

```
from lime.lime_tabular import LimeTabularExplainer

explainer = LimeTabularExplainer(
    X_train.values,
    feature_names=X.columns.tolist(),
    class_names=['no', 'yes'],
    mode='classification'
)
```

```
# Expliquer une prédiction
exp = explainer.explain_instance(
    X_test.values[0],
    model.predict_proba,
    num_features=10
)
exp.show_in_notebook()
exp.as_list() # liste (feature, weight)
```

## 11.5 Partial Dependence Plots (PDP) et ICE

---

PDP montre l'effet marginal moyen d'une feature sur la prédiction. ICE (Individual Conditional Expectation) montre l'effet pour chaque instance.

```
from sklearn.inspection import PartialDependenceDisplay

# PDP pour 1 feature
PartialDependenceDisplay.from_estimator(model, X_train, ['balance', 'age'])

# PDP 2D (interaction entre 2 features)
PartialDependenceDisplay.from_estimator(model, X_train, [('balance', 'age')])

# ICE curves (individual=True)
PartialDependenceDisplay.from_estimator(model, X_train, ['balance'],
                                         kind='individual')
```

## 11.6 Grad-CAM (CNN — images)

---

Grad-CAM visualise les régions de l'image qui ont influencé la décision du CNN. Utilise les gradients de la dernière couche convolutive.

```
# Gradient-weighted Class Activation Mapping
# pip install grad-cam
from pytorch_grad_cam import GradCAM
from pytorch_grad_cam.utils.image import show_cam_on_image

cam = GradCAM(model=model, target_layers=[model.layer4[-1]])
grayscale_cam = cam(input_tensor=img_tensor)
visualization = show_cam_on_image(img_np, grayscale_cam[0])
```

## 12. Clustering & Détection d'Anomalies

### 12.1 K-Means

Regroupe les données en k clusters en minimisant l'inertie (somme des distances au centroïde). Non supervisé.

```
from sklearn.cluster import KMeans

# Choisir k par la méthode du coude
inertias = []
for k in range(2, 11):
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X_scaled)
    inertias.append(km.inertia_)

# Ou par le score de silhouette
from sklearn.metrics import silhouette_score
sil_scores = []
for k in range(2, 11):
    km = KMeans(n_clusters=k, random_state=42)
    labels = km.fit_predict(X_scaled)
    sil_scores.append(silhouette_score(X_scaled, labels))
# Choisir k qui maximise le silhouette score (entre -1 et 1, plus grand = meilleur)
```

### 12.2 Clustering Hiérarchique

```
from sklearn.cluster import AgglomerativeClustering
import scipy.cluster.hierarchy as sch

# Dendrogramme pour visualiser la hiérarchie
dendrogram = sch.dendrogram(sch.linkage(X_scaled, method='ward'))

# Clustering final
ac = AgglomerativeClustering(n_clusters=4, linkage='ward')
labels = ac.fit_predict(X_scaled)
```

### 12.3 DBSCAN (density-based)

Trouve des clusters de forme arbitraire. Gère le bruit (outliers). Ne nécessite pas de spécifier k.

```
from sklearn.cluster import DBSCAN

# eps : rayon de voisinage, min_samples : points minimum pour un cluster
db = DBSCAN(eps=0.5, min_samples=10)
labels = db.fit_predict(X_scaled)
# labels = -1 → outliers/bruit
n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
```

### 12.4 Détection d'Anomalies

| Méthode              | Type                                                          | Code sklearn                                 |
|----------------------|---------------------------------------------------------------|----------------------------------------------|
| Isolation Forest     | Basé sur des arbres. Score d'anomalie = facilité d'isolation. | IsolationForest(contamination=0.05)          |
| Local Outlier Factor | Basé sur la densité locale.                                   | LocalOutlierFactor(n_neighbors=20)           |
| One-Class SVM        | Frontière autour des données normales.                        | OneClassSVM(nu=0.1, kernel='rbf')            |
| Autoencoder          | Reconstruction error élevée = anomalie.                       | PyTorch/Keras — architecture encoder-decoder |

Z-score

Statistique — suppose gaussianité.

`scipy.stats.zscore()`

## 13. Séries Temporelles

### 13.1 Caractéristiques d'une série temporelle

| Propriété        | Description                                      | Test / Détection                    |
|------------------|--------------------------------------------------|-------------------------------------|
| Stationnarité    | Moyenne, variance et autocorrélation constantes  | Test ADF (Augmented Dickey-Fuller)  |
| Tendance (trend) | Évolution long-terme                             | Décomposition STL, Hodrick-Prescott |
| Saisonnalité     | Variations périodiques à fréquence fixe (< 1 an) | FFT, décomposition saisonnière      |
| Cyclicité        | Variations à longue période sans fréquence fixe  | Analyse spectrale                   |
| Autocorrélation  | Corrélation avec les valeurs passées             | ACF (AutoCorrelation Function)      |

### 13.2 ARIMA et SARIMA

```
# ARIMA(p, d, q)
# p : ordre autorégressif (nombre de lags de y)
# d : degré de différenciation (0 si stationnaire, 1 sinon)
# q : ordre moyenne mobile (nombre de lags de l'erreur)

from statsmodels.tsa.arima.model import ARIMA
model = ARIMA(y_train, order=(2, 1, 2))
fitted = model.fit()
forecast = fitted.forecast(steps=30)

# SARIMA : ajoute des paramètres saisonniers (P, D, Q, s)
from statsmodels.tsa.statespace.sarimax import SARIMAX
model = SARIMAX(y, order=(2,1,2), seasonal_order=(1,1,1,12))

# Auto-ARIMA (cherche automatiquement les meilleurs ordres)
# pip install pmdarima
from pmdarima import auto_arima
model = auto_arima(y_train, seasonal=True, m=12)
```

### 13.3 Prophet (Facebook)

Modèle additif :  $y(t) = g(t) + s(t) + h(t) + \varepsilon(t)$ . Robuste aux données manquantes et aux outliers. Gère plusieurs saisonnalités.

```
from prophet import Prophet

df_prophet = pd.DataFrame({'ds': dates, 'y': values})
model = Prophet(seasonality_mode='multiplicative', yearly_seasonality=True)
model.fit(df_prophet)

future = model.make_future_dataframe(periods=365)
forecast = model.predict(future)
model.plot(forecast)
model.plot_components(forecast)
```

### 13.4 LSTM pour séries temporelles

```
# Créer les séquences (lookback window)
def create_sequences(data, window=30):
    X, y = [], []
```

```

    for i in range(len(data) - window):
        X.append(data[i:i+window])
        y.append(data[i+window])
    return np.array(X), np.array(y)

X_seq, y_seq = create_sequences(scaled_data, window=30)
# X_seq.shape = (n_samples, 30, n_features)

model = nn.Sequential(
    nn.LSTM(input_size=1, hidden_size=64, num_layers=2, batch_first=True),
    # Note : LSTM retourne (output, (h_n, c_n)) - utiliser Lambda pour extraire
)

```

## 14. Optimisation des Hyperparamètres

### 14.1 Stratégies de recherche

| Méthode               | Description                                                      | Quand utiliser                                                |
|-----------------------|------------------------------------------------------------------|---------------------------------------------------------------|
| Grid Search           | Teste toutes les combinaisons. Exhaustif.                        | Peu d'hyperparamètres ( $\leq 3$ ), petit espace de recherche |
| Randomized Search     | Échantillonnage aléatoire. Bien plus efficace.                   | Grand espace de recherche — toujours préférer à GridSearch    |
| Bayesian Optimization | Modèle probabiliste guide la recherche (Optuna, scikit-optimize) | Évaluation coûteuse (DL, grands datasets)                     |
| Halving Search        | Élimine progressivement les mauvais candidats                    | Compromis rapidité/qualité                                    |

```
# RandomizedSearchCV (recommandé)
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint, uniform

param_dist = {
    'n_estimators': randint(100, 500),
    'max_depth': randint(3, 15),
    'min_samples_leaf': randint(5, 50),
    'max_features': ['sqrt', 'log2', None],
}

search = RandomizedSearchCV(
    RandomForestClassifier(random_state=42),
    param_distributions=param_dist,
    n_iter=100,
    cv=StratifiedKFold(5),
    scoring='roc_auc',
    n_jobs=-1,
    random_state=42
)
search.fit(X_train, y_train)
print(search.best_params_, search.best_score_)

# Optuna (Bayesian Optimization — beaucoup plus efficace)
# pip install optuna
import optuna
def objective(trial):
    n_est = trial.suggest_int('n_estimators', 100, 1000)
    lr = trial.suggest_float('learning_rate', 0.01, 0.3, log=True)
    model = XGBClassifier(n_estimators=n_est, learning_rate=lr)
    return cross_val_score(model, X, y, cv=3, scoring='roc_auc').mean()

study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=100)
print(study.best_params_)
```



## 15. Cas d'Usage Complet — Bank Marketing Dataset

### Description du problème

Prédire si un client souscrit à un dépôt à terme ( $y=1$ ) ou non ( $y=0$ ) suite à une campagne téléphonique.

| Aspect          | Détail                                                  |
|-----------------|---------------------------------------------------------|
| Variable cible  | $y$ : 0 (non souscrit) ou 1 (souscrit)                  |
| Déséquilibre    | ~88% classe 0, ~12% classe 1 → ne pas utiliser accuracy |
| Métrique Kaggle | AUC-ROC                                                 |
| Taille          | ~45 000 train, 250 000 test                             |

### Features importantes

| Feature  | Description                                 | Note                                                                                        |
|----------|---------------------------------------------|---------------------------------------------------------------------------------------------|
| duration | Durée du dernier appel (secondes)           | Feature la plus prédictive MAIS connue seulement après l'appel → data leakage en production |
| poutcome | Résultat de la campagne précédente          | 'success' augmente fortement les chances                                                    |
| pdays    | Jours depuis dernier contact (-1 si jamais) | Créer un flag 'was_contacted_before'                                                        |
| month    | Mois du dernier contact                     | Mars, sept, oct, déc ont des taux plus élevés                                               |
| job      | Profession du client                        | Retraités et étudiants : plus susceptibles de souscrire                                     |
| balance  | Solde moyen annuel                          | Peut être négatif (découvert)                                                               |

### Pipeline complet recommandé

```
# =====  
# FEATURE ENGINEERING  
# =====  
df['was_contacted_before'] = (df['pdays'] != -1).astype(int)  
df['pdays'] = df['pdays'].replace(-1, 0)  
month_map = {'jan':1,'feb':2,'mar':3,'apr':4,'may':5,'jun':6,  
             'jul':7,'aug':8,'sep':9,'oct':10,'nov':11,'dec':12}  
df['month_num'] = df['month'].map(month_map)  
df['negative_balance'] = (df['balance'] < 0).astype(int)  
  
# =====  
# COLONNES  
# =====  
NUM = ['age', 'balance', 'duration', 'campaign', 'pdays',  
       'previous', 'month_num', 'was_contacted_before', 'negative_balance']  
CAT = ['job', 'marital', 'education', 'default', 'housing', 'loan',  
       'contact', 'poutcome']  
  
# =====  
# PREPROCESSOR  
# =====  
from sklearn.compose import ColumnTransformer  
from sklearn.preprocessing import StandardScaler, OrdinalEncoder
```

```

preprocessor = ColumnTransformer([
    ('num', StandardScaler(), NUM),
    ('cat', OrdinalEncoder(handle_unknown='use_encoded_value', unknown_value=-1), CAT)
])

# =====
# MODÈLES & ÉVALUATION
# =====

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.model_selection import StratifiedKFold, cross_val_score

neg, pos = (y==0).sum(), (y==1).sum()

models = {
    'LR' : LogisticRegression(class_weight='balanced', C=1, max_iter=1000),
    'RF' : RandomForestClassifier(n_estimators=200, class_weight='balanced',
                                max_depth=10, random_state=42),
    'XGB': XGBClassifier(n_estimators=300, learning_rate=0.05,
                        scale_pos_weight=neg/pos, random_state=42)
}

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
for name, clf in models.items():
    pipe = Pipeline([('pre', preprocessor), ('clf', clf)])
    auc = cross_val_score(pipe, X, y, cv=cv, scoring='roc_auc').mean()
    print(f'{name}: AUC = {auc:.4f}')

```

## Résultats attendus

| Modèle              | AUC indicatif  | Avantage                                             |
|---------------------|----------------|------------------------------------------------------|
| Logistic Regression | AUC ~0.79      | Baseline. Interprétable. Coefficients = odds ratios. |
| Random Forest       | AUC ~0.83      | Feature importance disponible. Robuste.              |
| XGBoost             | AUC ~0.85-0.87 | Meilleure performance. Après tuning.                 |

✓ Pour la présentation : insister sur la COMPRÉHENSION, pas le score.

Expliquer pourquoi 'duration' est problématique en production (leakage).

Montrer feature importance + discuter les limites du modèle.

## 16. Aide-Mémoire — Choisir le bon outil

### Choisir l'algorithme

| Algorithme          | Quand utiliser                                                      | Points d'attention                                            |
|---------------------|---------------------------------------------------------------------|---------------------------------------------------------------|
| Logistic Regression | Baseline, interprétabilité requise, données linéairement séparables | Normaliser les features, gérer les déséquilibres              |
| Random Forest       | Robustesse, peu de tuning, feature importance                       | Pas besoin de normalisation, gère bien les valeurs manquantes |
| XGBoost / LightGBM  | Meilleures performances sur données tabulaires                      | Plus de tuning nécessaire                                     |
| SVM                 | Haute dimension (texte), petits datasets                            | Obligatoirement normaliser                                    |
| MLP (réseau dense)  | Données tabulaires avec features complexes, beaucoup de données     | Normaliser, régulariser (Dropout, L2)                         |
| CNN                 | Images, texte (1D-Conv), données structurées spatialement           | Transfer Learning si peu de données                           |
| LSTM / GRU          | Séquences, séries temporelles, texte                                | Lent, difficile à entraîner sans GPU                          |
| BERT / Transformers | NLP — classification de texte, QA, génération                       | Lourd, nécessite GPU, fine-tuning                             |
| KMeans              | Clustering, exploration non supervisée                              | Choisir k, sensible aux outliers                              |
| Isolation Forest    | Détection d'anomalies                                               | Régler contamination                                          |

### Choisir la métrique

| Métrique  | Quand utiliser                                | Compromis                                 |
|-----------|-----------------------------------------------|-------------------------------------------|
| Accuracy  | Classes équilibrées, coûts égaux              | ÉVITER si déséquilibre                    |
| AUC-ROC   | Ranking des probabilités, déséquilibre        | Métrique par défaut Kaggle                |
| F1-score  | Équilibre precision/recall                    | Bon compromis général                     |
| Precision | Minimiser les faux positifs (spam, pub)       | Recall peut être faible                   |
| Recall    | Minimiser les faux négatifs (maladie, fraude) | Precision peut être faible                |
| Log-Loss  | Calibration des probabilités                  | Pénalise les erreurs confiantes           |
| RMSE      | Régression — sensible aux grosses erreurs     | Même unité que la cible                   |
| MAE       | Régression — robuste aux outliers             | Moins pénalisant pour les grosses erreurs |

## Normalisation — quand et comment

| Méthode              | Usage                                                                                 |
|----------------------|---------------------------------------------------------------------------------------|
| StandardScaler       | Moyenne 0, std 1. LR, SVM, PCA, KNN, réseaux de neurones. TOUJOURS.                   |
| MinMaxScaler         | Range [0,1]. Réseaux de neurones (ReLU). Si distribution non gaussienne.              |
| RobustScaler         | Utilise médiane et IQR. Résistant aux outliers.                                       |
| Pas de normalisation | Random Forest, XGBoost, arbres de décision. Invariants aux transformations monotones. |

## Erreurs fréquentes à éviter

⚠ 6 erreurs qui coûtent des points :

1. Data leakage : appliquer StandardScaler sur tout X avant le split. Utiliser TOUJOURS un Pipeline.
2. Mauvaise métrique : utiliser accuracy sur des données déséquilibrées. Préférer AUC-ROC ou F1.
3. Overfitting invisible : évaluer uniquement sur train. Utiliser cross-validation.
4. Hyperparamètres sur test set : choisir les hp en regardant le score test. Utiliser un validation set ou CV.
5. One-Hot sur ordinales : encoder 'education' en One-Hot perd l'information d'ordre.
6. Oublier random\_state : résultats non reproductibles. Toujours fixer random\_state=42.